

Sistema de Divisão de Despesas

Modelagem, tecnologias e plano de implementação de uma primeira versão web

Objetivo do documento

Organizar uma proposta prática para desenvolver um sistema web de divisão de despesas usando Python, Django, HTML, CSS, Bootstrap, JavaScript e banco de dados relacional.

Escopo: grupos de pessoas, registro de despesas, identificação de quem pagou, divisão entre participantes, cálculo de saldos e sugestão de transferências.

Documento de planejamento do MVP

Julho de 2026

Índice

1. Visão geral do sistema	4
2. Exemplo de funcionamento	4
3. Entidades principais	4
3.1 Usuário	4
3.2 Grupo	4
3.3 Participante do grupo	5
4. A entidade Despesa	5
5. Quem pagou a despesa	5
6. Quem participa da despesa	6
7. Formas de dividir uma despesa	6
7.1 Divisão igualitária	6
7.2 Evoluções futuras	6
8. Cálculo do saldo de cada pessoa	7
9. Modelo de banco de dados inicial	8
10. Fluxo básico da aplicação	8
11. Telas da primeira versão	9
11.1 Login e cadastro	9
11.2 Painel principal	9
11.3 Tela do grupo	9
11.4 Nova despesa	9
11.5 Detalhes da despesa	9
11.6 Tela de acerto	9
12. Tecnologias Python recomendadas	9

12.1 Django	9
12.2 Banco de dados	9
12.3 ORM do Django	9
12.4 Valores monetários	10
13. Tecnologias auxiliares	10
13.1 HTML	10
13.2 CSS e Bootstrap	10
13.3 JavaScript	10
13.4 API	10
14. Organização sugerida do projeto Django	11
15. Separação entre regras e interface	11
16. Regras importantes do sistema	11
17. O problema dos centavos	12
18. Sugestão de transferências	12
19. Escopo recomendado para o MVP	13
19.1 Incluir	13
19.2 Deixar para depois	13
20. Ordem prática de desenvolvimento	14
21. Stack recomendada	14
22. Modelo conceitual resumido	14

1. Visão geral do sistema

A ideia central está correta: haverá despesas pagas por uma ou mais pessoas e essas despesas serão divididas entre os participantes de um grupo. Para que a aplicação permaneça organizada e possa evoluir, é importante separar três conceitos:

- Despesa: o gasto realizado, como restaurante, combustível ou hospedagem.
- Pagamento: o dinheiro efetivamente desembolsado por uma pessoa.
- Divisão: a parte da despesa que cabe a cada participante.

Decisão de modelagem: a despesa deve ser o elemento central. O pagamento registra quem colocou o dinheiro; a participação registra quem deve suportar o custo.

2. Exemplo de funcionamento

Considere uma viagem com Ana, Bruno, Carlos e Daniela. O grupo registra várias despesas:

Despesa	Valor	Quem pagou	Quem participa
Restaurante	R\$ 200,00	Ana	Todos
Combustível	R\$ 160,00	Bruno	Todos
Passeio	R\$ 180,00	Carlos	Ana, Carlos e Daniela

O sistema deverá responder quanto foi gasto, quanto cada pessoa deveria pagar, quanto cada uma realmente pagou, quem deve receber, quem deve pagar e quais transferências encerram as contas.

3. Entidades principais

3.1 Usuário

Representa uma pessoa que possui acesso ao sistema. O Django já fornece uma estrutura de autenticação que pode ser aproveitada.

```
Usuário
- id
- nome
- email
- senha_hash
- data_criacao
```

A senha nunca deve ser armazenada diretamente. O sistema guarda apenas um hash seguro.

3.2 Grupo

Representa o contexto da divisão: uma viagem, uma família, um evento ou qualquer conjunto de despesas compartilhadas.

```
Grupo
- id
- nome
- descricao
- criado_por
- data_criacao
- status
```

O status pode ser ativo, encerrado ou arquivado.

3.3 Participante do grupo

Um grupo possui vários participantes e um usuário pode participar de vários grupos. Essa é uma relação muitos-para-muitos.

```
ParticipanteGrupo
- id
- grupo_id
- usuario_id
- nome_exibicao
- papel
- ativo
```

Para facilitar o uso, a primeira versão pode permitir participantes sem cadastro completo, adicionados apenas pelo nome.

4. A entidade Despesa

A despesa reúne as informações do gasto. Ela pertence a um único grupo e deve registrar valor, data, descrição e demais dados necessários.

```
Despesa
- id
- grupo_id
- descricao
- valor_total
- data_despesa
- categoria
- observacao
- criado_por
- criado_em
```

Exemplo: Jantar, R\$ 240,00, em 20/07/2026, categoria Alimentação, grupo Viagem para Recife.

5. Quem pagou a despesa

Para o MVP, uma despesa pode ter apenas um pagador. Mesmo assim, é recomendável usar uma tabela separada para o pagamento, pois isso prepara o sistema para aceitar vários pagadores futuramente.

```
Pagamento
- id
- despesa_id
- participante_id
- valor_pago
```

No futuro, a mesma despesa poderá registrar, por exemplo, Ana pagando R\$ 150,00 e Bruno pagando R\$ 90,00.

6. Quem participa da despesa

Nem todos os membros do grupo precisam participar de todas as despesas. Por isso, a aplicação deve registrar os participantes específicos de cada gasto.

```
ParticipacaoDespesa
- id
- despesa_id
- participante_id
- valor_devido
```

Participante	Valor devido
Ana	R\$ 60,00
Bruno	R\$ 60,00
Carlos	R\$ 60,00
Daniela	R\$ 60,00

A soma dos valores devidos deve ser exatamente igual ao valor total da despesa.

7. Formas de dividir uma despesa

7.1 Divisão igualitária

É a única forma necessária para a primeira versão:

$$\text{valor individual} = \text{valor total} \div \text{número de participantes}$$

Exemplo: R\$ 200,00 divididos entre quatro pessoas resultam em R\$ 50,00 para cada uma.

7.2 Evoluções futuras

- Valor específico: cada pessoa recebe um valor definido manualmente.
- Porcentagem: cada participante assume uma parcela percentual.
- Cotas ou pesos: adultos podem ter peso 2 e crianças peso 1.

Recomendação para o MVP: implementar somente a divisão igualitária entre participantes selecionados.

8. Cálculo do saldo de cada pessoa

O saldo individual resulta da diferença entre o total pago e o total devido:

$$\text{saldo} = \text{total pago} - \text{total devido}$$

- Saldo positivo: a pessoa deve receber.
- Saldo negativo: a pessoa deve pagar.
- Saldo zero: a pessoa está quitada.

Pessoa	Total pago	Total devido	Saldo
Ana	R\$ 300,00	R\$ 150,00	R\$ 150,00
Bruno	R\$ 50,00	R\$ 150,00	-R\$ 100,00
Carlos	R\$ 100,00	R\$ 150,00	-R\$ 50,00

Resultado: Bruno paga R\$ 100,00 para Ana e Carlos paga R\$ 50,00 para Ana.

9. Modelo de banco de dados inicial

```
Usuario
- id
- nome
- email
- senha_hash

Grupo
- id
- nome
- descricao
- criado_por
- criado_em
- status

ParticipanteGrupo
- id
- grupo_id
- usuario_id
- nome_exibicao
- papel
- ativo

Despesa
- id
- grupo_id
- descricao
- valor_total
- data_despesa
- categoria
- observacao
- criado_por
- criado_em

Pagamento
- id
- despesa_id
- participante_id
- valor_pago

ParticipacaoDespesa
- id
- despesa_id
- participante_id
- valor_devido
```

Relações principais:

```
Usuário 1 --- N Grupo
Grupo 1 --- N ParticipanteGrupo
Grupo 1 --- N Despesa
Despesa 1 --- N Pagamento
Despesa 1 --- N ParticipacaoDespesa
```

10. Fluxo básico da aplicação

- 1. Criar uma conta: nome, e-mail e senha.
- 2. Criar um grupo: nome e descrição.

- 3. Adicionar participantes: usuários cadastrados ou nomes convidados.
- 4. Registrar uma despesa: descrição, valor, data e pagador.
- 5. Selecionar participantes: escolher quem dividirá aquela despesa.
- 6. Dividir igualmente: o sistema calcula o valor devido por pessoa.
- 7. Visualizar saldos: identificar credores e devedores.
- 8. Visualizar acertos: apresentar as transferências sugeridas.

11. Telas da primeira versão

11.1 Login e cadastro

Campos de e-mail, senha, entrada e criação de conta.

11.2 Painel principal

Lista de grupos e resumo do total que o usuário deve pagar ou receber.

11.3 Tela do grupo

Participantes, total gasto, despesas, saldos e botão para adicionar despesa.

11.4 Nova despesa

Descrição, valor, data, categoria, pagador, participantes, forma de divisão e observação.

11.5 Detalhes da despesa

Exibe os dados completos e permite editar ou excluir.

11.6 Tela de acerto

Mostra quem deve pagar, quem deve receber e as transferências sugeridas.

12. Tecnologias Python recomendadas

12.1 Django

Django é a melhor escolha inicial porque já inclui autenticação, sessões, ORM, formulários, painel administrativo, proteção contra ataques comuns e migrations.

12.2 Banco de dados

- Desenvolvimento: SQLite.
- Produção: PostgreSQL.

12.3 ORM do Django

O ORM permite representar tabelas como classes Python:

```
class Grupo(models.Model):  
    nome = models.CharField(max_length=100)  
    descricao = models.TextField(blank=True)
```

12.4 Valores monetários

Valores financeiros devem usar Decimal, nunca float.

```
from decimal import Decimal
valor = Decimal("100.00")

valor_total = models.DecimalField(
    max_digits=10,
    decimal_places=2,
)
```

13. Tecnologias auxiliares

13.1 HTML

Estrutura das páginas: formulários, tabelas, listas, campos e botões.

13.2 CSS e Bootstrap

Responsáveis pelo visual e pela adaptação para celular. Bootstrap acelera a criação de uma interface organizada.

13.3 JavaScript

Pode recalcular a divisão antes do envio, selecionar participantes, validar totais e melhorar a interação. No início, use apenas o necessário.

13.4 API

Uma API não é obrigatória no MVP. Ela pode ser criada futuramente com Django REST Framework para aplicativo móvel ou frontend separado.

14. Organização sugerida do projeto Django

```
divisao_despesas/  
├── manage.py  
├── config/  
│   ├── settings.py  
│   ├── urls.py  
│   └── wsgi.py  
├── usuarios/  
│   ├── models.py  
│   ├── views.py  
│   ├── forms.py  
│   └── urls.py  
├── grupos/  
│   ├── models.py  
│   ├── views.py  
│   ├── forms.py  
│   └── urls.py  
├── despesas/  
│   ├── models.py  
│   ├── views.py  
│   ├── forms.py  
│   ├── services.py  
│   └── urls.py  
├── templates/  
├── static/  
│   ├── css/  
│   └── js/  
└── tests/
```

A lógica de cálculo deve ficar em `services.py` ou em outro módulo independente, e não diretamente nas `views`.

15. Separação entre regras e interface

A regra de negócio deve funcionar sem depender de HTML ou do banco. Isso facilita testes, correções, reutilização e criação de uma API.

```
def calcular_saldos(pagamentos, valores_devidos):  
    saldos = {}  
    pessoas = pagamentos.keys() | valores_devidos.keys()  
  
    for pessoa in pessoas:  
        total_pago = pagamentos.get(pessoa, Decimal("0.00"))  
        total_devido = valores_devidos.get(pessoa, Decimal("0.00"))  
        saldos[pessoa] = total_pago - total_devido  
  
    return saldos
```

16. Regras importantes do sistema

- O valor da despesa deve ser maior que zero.
- Uma despesa deve possuir pelo menos um participante.

- O pagador deve pertencer ao grupo.
- Os participantes da despesa devem pertencer ao grupo.
- A soma dos pagamentos deve ser igual ao valor total.
- A soma dos valores devidos deve ser igual ao valor total.
- Uma despesa pertence a apenas um grupo.
- Somente pessoas autorizadas podem visualizar ou alterar o grupo.
- Alterações em despesas exigem recálculo dos saldos.
- Diferenças de centavos devem ser tratadas de forma determinística.

17. O problema dos centavos

R\$ 100,00 divididos por três resultam em R\$ 33,333... A aplicação precisa distribuir os centavos restantes sem perder a igualdade do total.

Pessoa	Valor
Pessoa 1	R\$ 33,34
Pessoa 2	R\$ 33,33
Pessoa 3	R\$ 33,33
Total	R\$ 100,00

```
from decimal import Decimal, ROUND_DOWN

def dividir_igualmente(valor_total, participantes):
    quantidade = len(participantes)
    if quantidade == 0:
        raise ValueError("A despesa precisa ter participantes.")

    valor_base = (valor_total / quantidade).quantize(
        Decimal("0.01"), rounding=ROUND_DOWN
    )

    divisoes = {p: valor_base for p in participantes}
    restante = valor_total - (valor_base * quantidade)
    centavos = int(restante / Decimal("0.01"))

    for participante in participantes[:centavos]:
        divisoes[participante] += Decimal("0.01")

    return divisoes
```

18. Sugestão de transferências

Depois dos saldos, o sistema separa credores e devedores. Em seguida, transfere valores entre eles até todos os saldos chegarem a zero.

```
credores = [{"Ana", Decimal("150.00")}]

devedores = [
    ["Bruno", Decimal("100.00")],
    ["Carlos", Decimal("50.00")],
]
```

Resultado:
Bruno paga R\$ 100,00 para Ana.
Carlos paga R\$ 50,00 para Ana.

No início, essas sugestões podem ser calculadas sempre que a tela for aberta, sem necessidade de armazená-las no banco.

19. Escopo recomendado para o MVP

19.1 Incluir

- Cadastro e login.
- Criação de grupos e participantes.
- Criação, edição e exclusão de despesas.
- Um pagador por despesa.
- Divisão igualitária.
- Seleção dos participantes de cada despesa.
- Cálculo de saldos e sugestão de transferências.
- Histórico de despesas.
- Layout responsivo.

19.2 Deixar para depois

- Vários pagadores na mesma despesa.
- Moedas diferentes.
- Divisão por porcentagem ou cotas.
- Anexos e leitura automática de comprovantes.
- WhatsApp, Pix e integração bancária.
- Aplicativo móvel, React e funcionamento offline.
- Despesas recorrentes.

20. Ordem prática de desenvolvimento

Etapa	Objetivo
1. Cálculo isolado	Criar funções Python para divisão, centavos, saldos e transferências; escrever testes.
2. Projeto Django	Criar projeto, apps, modelos, migrations e painel administrativo.
3. Grupos	Criar, listar e administrar grupos e participantes.
4. Despesas	Cadastrar despesa, pagador, participantes e divisão igualitária.
5. SalDOS	Exibir total pago, devido e saldo individual.
6. Acertos	Gerar a lista de transferências sugeridas.
7. Produção	Permissões, PostgreSQL, variáveis de ambiente, HTTPS, logs e backup.

21. Stack recomendada

Python 3.12 ou 3.13
Django
SQLite no desenvolvimento
PostgreSQL na produção
HTML
CSS
Bootstrap
JavaScript
Git e GitHub
Testes do Django ou pytest
Docker posteriormente

Arquitetura inicial recomendada:

Frontend com templates HTML do Django; backend Django; Bootstrap para a interface; JavaScript apenas onde necessário; SQLite durante o desenvolvimento e PostgreSQL na publicação.

22. Modelo conceitual resumido

Um usuário cria um grupo.

O grupo possui participantes.

O grupo possui várias despesas.

Cada despesa possui:

- um valor;
- um ou mais pagamentos;
- participantes;
- valores devidos.

O sistema soma quanto cada participante pagou e quanto deveria pagar.

A diferença gera o saldo.

Os saldos geram as transferências sugeridas.

Conclusão: comece com Django, templates HTML, Bootstrap e funções Python independentes para as regras de divisão. Concentre o MVP em grupos, despesas, um pagador por despesa, divisão igualitária, saldos e acertos. Essa base é simples o suficiente para aprender e sólida o bastante para crescer.